

FH Aachen

Bachelor's Thesis

in partial fulfillment of the requirements for the degree of
B.Sc. in Computer Science

**AI Agents in Software Development: Best Practices for
Structured Human-Agent Collaboration**

Fakih Lanjri Houdaifa

Matriculation Number 3574047 Aachen, First Examiner:
Second Examiner:

Declaration

I hereby declare that I have written this thesis independently and have not used any sources other than those indicated in the bibliography. Passages that have been taken verbatim or in substance from published or as yet unpublished sources are identified as such. The drawings or illustrations in this thesis were created by myself or are provided with an appropriate reference to their source. This thesis has not been submitted in the same or similar form to any other examination authority.

Aachen,

Contents

- 1 Introduction 7**
 - 1.1 Background 7
 - 1.2 Motivation 7
 - 1.3 Problem Statement 7
 - 1.4 Objectives of the Thesis 7
- 2 Foundations 9**
 - 2.1 Large Language Models 9
 - 2.2 From LLMs to AI Agents 9
 - 2.3 Claude Code as a Coding Agent 9
 - 2.4 Prompt Engineering 9
 - 2.5 Multi-Agent Systems 9
 - 2.6 Software Engineering Foundations 9
 - 2.7 Human-AI Collaboration 9
- 3 Methodology 11**
 - 3.1 Structure of the Practical Project (Two Sub-Projects) 11
 - 3.2 Tools and Models Used 11
 - 3.3 Evaluation Criteria 11
- 4 Part 1: Unstructured Development 13**
 - 4.1 Initial Situation and Objectives 13
 - 4.2 Prompting Without a Structural Approach 13
 - 4.3 System Prompt Context 13
 - 4.4 Development of the First Feature (Login/Registration) 13
 - 4.5 Test Suites and Test Case Generation 13
 - 4.6 Architectural Analysis of the Generated Software 13
 - 4.7 Observations: Prompt Precision, Human Oversight, Architectural Limitations . 13
 - 4.8 Critical Problems and the Test Failure Analysis Principle” 13
- 5 Part 2: Structured Approach (AntiGravity) 15**
 - 5.1 Motivation for a Structured Approach 15
 - 5.2 AI Development Charter and Architectural Rules 15

5.3 Parallel Agent Architecture (Architect / Implementer / Post-Implementation Review Agent)	15
5.4 Design Reviews Before Coding (implementation_plan.md)	15
5.5 Testing Strategy: Force Requirement Validation & Three-Tier Testing	15
5.6 Project Knowledge Files	15
5.7 OpenAPI Specification Automation and Avoiding Documentation Drift	15
5.8 Identification of Architectural Problems and Weaknesses	15
6 Comparison of Both Approaches	17
6.1 Comparison: Part 1 vs. Part 2	17
6.2 Feature Diversity	17
7 Evaluation	19
7.1 Evaluation Criteria and Methodology	19
7.2 Quality of Generated Code	19
7.3 Quality and Validity of Generated Tests	19
7.4 Architectural Conformance and Documentation Consistency	19
7.5 Efficiency and Effort (Human vs. Agent)	19
7.6 Summary Assessment: Part 1 vs. Part 2	19
8 Best Practices for Human-Agent Collaboration	21
8.1 Importance of Prompt Precision and Context Design	21
8.2 Role of Human Oversight (Human-in-the-Loop)	21
8.3 Architectural Guardrails for AI Agents	21
8.4 Recommendations for Multi-Agent Workflows	21
8.5 Avoiding Documentation Drift	21
9 Discussion	23
9.1 Opportunities and Limitations of AI Agents in Software Development	23
9.2 Implications for Practice	23
10 Conclusion and Outlook	25
Bibliography	27

1 Introduction

1.1 Background

1.2 Motivation

1.3 Problem Statement

1.4 Objectives of the Thesis

2 Foundations

2.1 Large Language Models

2.2 From LLMs to AI Agents

2.3 Claude Code as a Coding Agent

2.4 Prompt Engineering

2.5 Multi-Agent Systems

2.6 Software Engineering Foundations

2.7 Human-AI Collaboration

3 Methodology

3.1 Structure of the Practical Project (Two Sub-Projects)

3.2 Tools and Models Used

3.3 Evaluation Criteria

4 Part 1: Unstructured Development

4.1 Initial Situation and Objectives

4.2 Prompting Without a Structural Approach

4.3 System Prompt Context

4.4 Development of the First Feature (Login/Registration)

4.5 Test Suites and Test Case Generation

4.6 Architectural Analysis of the Generated Software

4.7 Observations: Prompt Precision, Human Oversight, Architectural Limitations

4.8 Critical Problems and the Test Failure Analysis Principle”

5 Part 2: Structured Approach (AntiGravity)

5.1 Motivation for a Structured Approach

5.2 AI Development Charter and Architectural Rules

5.3 Parallel Agent Architecture (Architect / Implementer / Post-Implementation Review Agent)

5.4 Design Reviews Before Coding (implementation_plan.md)

5.5 Testing Strategy: Force Requirement Validation & Three-Tier Testing

5.6 Project Knowledge Files

5.7 OpenAPI Specification Automation and Avoiding Documentation Drift

5.8 Identification of Architectural Problems and Weaknesses

6 Comparison of Both Approaches

6.1 Comparison: Part 1 vs. Part 2

6.2 Feature Diversity

7 Evaluation

7.1 Evaluation Criteria and Methodology

7.2 Quality of Generated Code

7.3 Quality and Validity of Generated Tests

7.4 Architectural Conformance and Documentation Consistency

7.5 Efficiency and Effort (Human vs. Agent)

7.6 Summary Assessment: Part 1 vs. Part 2

8 Best Practices for Human-Agent Collaboration

8.1 Importance of Prompt Precision and Context Design

8.2 Role of Human Oversight (Human-in-the-Loop)

8.3 Architectural Guardrails for AI Agents

8.4 Recommendations for Multi-Agent Workflows

8.5 Avoiding Documentation Drift

9 Discussion

9.1 Opportunities and Limitations of AI Agents in Software Development

9.2 Implications for Practice

10 Conclusion and Outlook

Bibliography